



PROS-NC-003

PROCEDIMIENTO DE DESARROLLO SEGURO

Sistema de Gestión de Seguridad de la Información – Nivel Central

Versión Oficial Actual v01 – Octubre del 2019

	Responsable	Fecha	Firma
Elaborado	Rodrigo Vidal / Encargado PMG SSI	Octubre 2019	
Revisado	José Villa / Área Seguridad de la Información (Representante Comité de Seguridad)	Octubre 2019	
Aprobado	Gabriel Reveco / Encargado Ciberseguridad (Presidente Comité de Seguridad de la Información)	Octubre 2019	



Contenido

1	PROPÓSITO	4
2	ALCANCE	4
3	TERMINOLOGÍA.....	4
4	DOCUMENTOS APLICABLES	4
5	ROLES Y RESPONSABILIDADES	4
6	PROCEDIMIENTO.....	5
6.1	ARQUITECTURA DE LA PLATAFORMA DE DESARROLLO	5
6.2	METODOLOGIA DE DESARROLLO	5
6.2.1	El método ágil versus el modelo de cascada.....	5
6.2.2	Decisión sobre que metodología que se debe utilizar	5
6.3	CONTROLES PROACTIVOS EN DESARROLLO DE SOFTWARE REQUERIDOS	6
6.3.1	¿Cuándo probar o realizar el testing?.....	6
6.3.2	Verificar la seguridad desde el inicio.....	6
6.3.3	Pruebas de Seguridad según OWASP	7
6.4	ESTANDARES DE CODIFICACION SEI CERT A CONSIDERAR	12
6.4.1	Validar la entrada	12
6.4.2	Prestar atención a las advertencias del compilador	12
6.4.3	Arquitectura y diseño de políticas de seguridad	12
6.4.4	Mantenlo simple	12
6.4.5	Por defecto denegar	12
6.4.6	Cumplir con el principio de privilegio mínimo.....	12
6.4.7	Sanitizar los datos enviados a otros sistemas	13
6.4.8	Practicar la defensa en profundidad.....	13
6.4.9	Usar técnicas efectivas de aseguramiento de la calidad	13
6.4.10	Adoptar una norma de codificación segura.....	13
6.4.11	Definir los requisitos de seguridad.....	13
6.4.12	Modelo de amenazas	14
6.5	OWASP PROYECTO DE CICLO DE VIDA DE DESARROLLO SEGURO (S-SDLC)	14
6.6	LAS 10 MEJORES PRÁCTICAS DE OWASP	14
6.6.1	Inyección.....	14
6.6.2	Pérdida de Autenticación	14
6.6.3	Exposición de datos sensibles	15
6.6.4	Entidades Externas XML (XXE).....	15
6.6.5	Pérdida de Control de Acceso.....	15
6.6.6	Configuración de Seguridad Incorrecta	15
6.6.7	Secuencia de Comandos en Sitios Cruzados (XSS).....	15
6.6.8	Deserialización Insegura.....	15

6.6.9	Componentes con vulnerabilidades conocidas	15
6.6.10	Registro y Monitoreo Insuficientes.....	16
6.7	ENTORNOS DE DESARROLLO SEPARADOS DE PRODUCCIÓN.....	16
6.8	DOCUMENTACION REQUERIDA SOBRE DESARROLLOS.....	17
6.8.1	Modelo de Datos	17
6.8.2	Modelo Entidad Relación	17
6.8.3	Documentación de Sistemas (Diagrama de diseño Lógico).....	17
6.8.4	Documentación de Requisitos Básicos.....	18
6.8.5	Manuales de Usuarios.....	18
6.9	IMPLEMENTACION DE DESARROLLOS ADQUIRIDOS A TERCEROS	18
6.10	REQUISITO PARA PASO A PRODUCCION.....	18
7	REGISTROS.....	19
8	DIFUSION	19
9	REVISION Y MEDICION	19
10	CONTROL DE VERSIONES	19
11	ANEXOS	20
Anexo 1: directrices ISO 27001 a cumplir.....		20
Anexo 2: listado de herramientas que pueden sr utilizadas para test de aplicaciones.....		20

1 PROPÓSITO

Establecer un marco regulatorio mínimo para el desarrollo de proyectos y aplicaciones tecnológicas de Minsal, con el propósito de que estos desarrollos estén alineados con los procedimientos de seguridad de la información definidos, durante todo el ciclo de vida de los sistemas.

2 ALCANCE

Las áreas de desarrollo de la organización y todas las áreas usuarias de las aplicaciones desarrolladas.

Subsecretarías de Salud Pública y de Redes Asistenciales.

Este procedimiento es aplicable a todos los funcionarios¹ (planta, contrata, reemplazos y suplencia), personal a honorarios y terceros (proveedores, compra de servicios, etc.), que presten servicios para las Subsecretarías de Salud Pública y de Redes Asistenciales y que tengan derechos de acceso a la información que puedan afectar los activos de información del Ministerio de Salud.

Este procedimiento abarca los siguientes controles definidos en la norma NCh-ISO 27001.Of2013:

- A.12.01.04 Separación de los ambientes de desarrollo, prueba y operacionales
- A.12.02.01 Controles contra código malicioso
- A.12.05.01 Instalación del software en sistemas operacionales
- A.12.06.02 Restricciones sobre la instalación de software
- A.14.02.02 Procedimientos de control de cambios del sistema
- A.14.02.06 Entorno de desarrollo seguro
- A.14.02.08 Prueba de seguridad del sistema
- A.14.02.09 Prueba de aprobación del sistema

3 TERMINOLOGÍA

MINSAL: Ministerio de Salud.

SGSI: Sistema de Gestión de Seguridad de Información.

4 DOCUMENTOS APLICABLES

- NCh-ISO27001.Of2013 Tecnología de la información – Técnicas de seguridad – Sistemas de gestión de la seguridad de la información – Requisitos.

5 ROLES Y RESPONSABILIDADES

Todas las áreas de desarrollo de Minsal, los Usuarios, Custodios Físicos, Custodios de los Datos, contratistas, Administradores de Seguridad, las Unidades de Informática, Gestión de Personas y el Comité de Seguridad de la Información, son responsables del cumplimiento de este procedimiento.

¹ A lo largo del procedimiento cada vez que se mencione funcionario se refiere a: funcionarios (planta, contrata, reemplazos y suplencia), personal a honorarios y terceros (proveedores, compra de servicios, etc.).

6 PROCEDIMIENTO

6.1 ARQUITECTURA DE LA PLATAFORMA DE DESARROLLO

Las aplicaciones para desarrollar deberán considerar la compatibilidad con el software base administrativo o institucional y/o las aplicaciones ya existentes como así mismo las características de las redes de datos existentes.

6.2 METODOLOGIA DE DESARROLLO

6.2.1 El método ágil versus el modelo de cascada

El método de cascada es un flujo de trabajo más tradicional y lineal. Utiliza un flujo de trabajo secuencial para gestionar las tareas. Cada fase de desarrollo está muy planificada y la fase anterior debe ser completada antes de pasar a la siguiente. Por ejemplo, la fase de planificación de necesidades debe estar completa antes de pasar a la fase de diseño. La fase de diseño debe entonces ser completada antes de pasar a la fase de implementación, la cual debe ser terminada antes de pasar a la fase de prueba. El método de la cascada puede ser bueno para abordar proyectos masivos que necesitan ser desglosados y distribuidos entre equipos más pequeños. Sin embargo, puede ser torpe, los retrasos o problemas en una fase pueden ondular durante todo el ciclo de vida del proyecto.

El modelo ágil busca eliminar este tipo de flujo de trabajo basado en escenarios, este se centra en la colaboración y la funcionalidad cruzada. Los desarrolladores ágiles son adaptables y enfocados en la mejora continua. En lugar de segmentar los proyectos en etapas, el desarrollo ágil tiende a abordar los proyectos en su conjunto. Los lugares de trabajo ágiles tienden a tener un equipo pequeño, pero altamente capacitado que puede manejar de manera fluida todas las etapas de implementación (es decir, planificación, codificación y pruebas) a la vez. Bajo el modelo ágil, las actualizaciones de software pequeñas pero completas se revelan con frecuencia. Bajo el método de la cascada, los nuevos productos o las actualizaciones pueden tardar más, pero pueden también ser mucho más grandes y más complejos.

6.2.2 Decisión sobre que metodología que se debe utilizar

La decisión sobre que metodología utilizar dependerá básicamente del tamaño del proyecto que se desea desarrollar y del enfoque que el área de desarrollo tenga implementado para su equipo de trabajo. El modelo ágil busca eliminar a muchos especialistas en un área de conocimiento, dando espacio a pocos desarrolladores con habilidades más amplias. Por esta razón, el modelo ágil tiende a funcionar mejor en pequeñas o medianas organizaciones que tienen equipos más compactos. Las organizaciones con proyectos a gran escala que no pueden ser manejados por un equipo pequeño a menudo encuentran que un modelo verdaderamente ágil es difícil de implementar, aunque pueden seguir ciertos aspectos de la metodología. Para efectos de este documento será decisión del área de desarrollo que metodología utilizará.

6.3 CONTROLES PROACTIVOS EN DESARROLLO DE SOFTWARE REQUERIDOS

6.3.1 ¿Cuándo probar o realizar el testing?

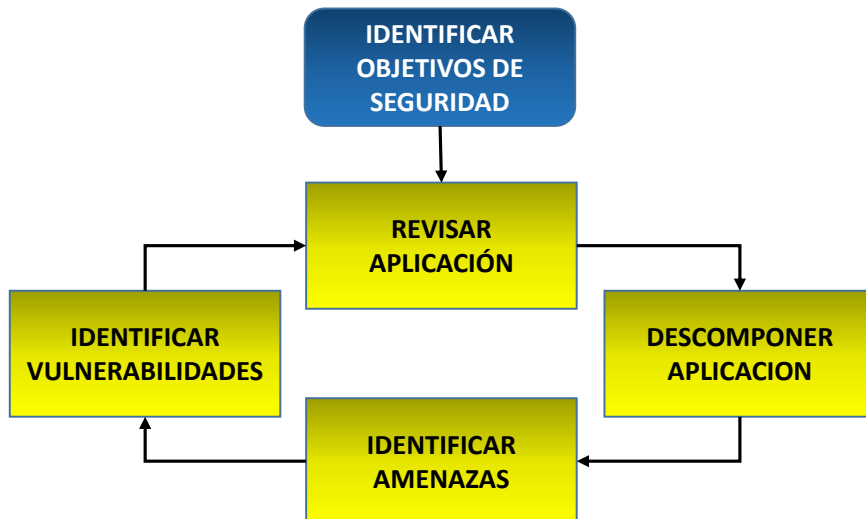
Uno de los mejores métodos para evitar que aparezcan errores de seguridad en las aplicaciones de producción es mejorar el Ciclo de vida de desarrollo de software (SDLC) al incluir seguridad en cada una de sus fases. Un SDLC es una estructura impuesta en el desarrollo de sistemas. Por lo tanto, se recomienda realizar pruebas en las etapas de Análisis, Diseño e Implementación y no solo en la etapa de Pruebas.

6.3.2 Verificar la seguridad desde el inicio

Las pruebas de seguridad deben ser parte integral de la práctica de ingeniería de software de un desarrollador. Del mismo modo que no puede “probar la calidad”, no puede “probar la seguridad” realizando pruebas de seguridad al final de un proyecto. Se debe verificar la seguridad de manera temprana y lo más frecuentemente posible, ya sea mediante pruebas manuales o pruebas y escaneos automatizados.

Ud. deberá considerar OWASP ASVS (Estándares de verificación de seguridad en aplicaciones) como una guía para definir los requisitos de seguridad y las pruebas. Debe considerar las protecciones de datos desde el inicio del desarrollo. La seguridad debe ser incluida por adelantado cuando se defina la estructura, no hacerlo implica si o si, problemas seguros posteriormente.

Modelo de Prueba:



6.3.3 Pruebas de Seguridad según OWASP

- Recopilación de Información
- Pruebas de gestión de la configuración
- Pruebas de la lógica de negocio
- Pruebas de Autenticación
- Pruebas de Autorización
- Pruebas de gestión de sesiones
- Pruebas de validación de datos
- Pruebas de denegación de Servicio
- Pruebas de Servicios Web
- Pruebas de AJAX
- Pruebas de Criptografías
- Manejo de Errores
- Protección de Datos
- Código Malicioso

a) Recopilación de Información

La primera fase en la evaluación de seguridad se centra en recoger tanta información como sea posible sobre una aplicación objetivo. La recopilación de información es un paso necesario en una prueba de intrusión. Esta tarea se puede llevar a cabo de muchas formas.

- ✓ Spiders, Robots, y Crawlers
- ✓ Reconocimiento mediante motores de Búsqueda
- ✓ Identificación de puntos de entrada de la aplicación
- ✓ Pruebas para encontrar firmas de Aplicaciones Web
- ✓ Descubrimiento de aplicaciones
- ✓ Análisis de códigos de error

b) Pruebas de Gestión de la Configuración

A menudo los análisis sobre la infraestructura o la topología de la arquitectura pueden revelar datos importantes sobre una aplicación Web. Se pueden obtener datos como por ejemplo el código fuente, los métodos HTTP permitidos, funcionalidades administrativas, métodos de autenticación y configuraciones de la infraestructura.

- ✓ Pruebas de SSL/TLS
- ✓ Pruebas del receptor de escucha de la BD
- ✓ Pruebas de gestión de configuración de la infraestructura
- ✓ Pruebas de gestión de configuración de la aplicación
- ✓ Gestión de extensiones de archivo
- ✓ Archivos antiguos, copias de seguridad y sin referencias
- ✓ Interfaces de administración de la infraestructura y de la aplicación
- ✓ Métodos HTTP y XST

c) Pruebas de Lógica de Negocio

Comprobar por fallas en la lógica de negocio en una aplicación web multifuncional requiere pensar en modos no convencionales. ¿Si el mecanismo de autenticación de una aplicación es desarrollado con la intención de seguir pasos 1,2,3 para poder autenticarse, que pasa si uno salta del paso 1 directo al 3? En este ejemplo, la aplicación o bien provee acceso fallando el mecanismo de autenticación, muestra un mensaje de error de acceso negado, o solo un mensaje de error 500.

La aplicación verificada debe satisfacer los siguientes requisitos de alto nivel:

- ✓ El flujo lógico de negocio es secuencial y en orden
- ✓ La lógica de negocio incluye límites para detectar y prevenir ataques automatizados, tales como pequeñas transferencias de fondos continuas o transferencias masivas a múltiples usuarios, etcétera.
- ✓ Los flujos de lógica de negocios de alto valor han considerado los casos de abuso y los actores maliciosos, y tienen protecciones contra el engaño, la manipulación, el repudio, la divulgación de información y los ataques de elevación de privilegios.

d) Comprobación del Sistema de Autenticación

Autenticar un objeto puede significar confirmar su procedencia, mientras que autenticar a una persona consiste a menudo en verificar su identidad. La autenticación depende de uno o más factores de autenticación. En seguridad informática, autenticación es el proceso de intentar verificar la identidad digital del remitente de una comunicación.

- ✓ Transmisión de credenciales a través de un canal cifrado
- ✓ Enumeración de usuarios
- ✓ Pruebas de diccionario sobre cuentas de Usuario o cuentas predeterminadas
- ✓ Pruebas de Fuerza Bruta
- ✓ Saltarse el sistema de autenticación
- ✓ Comprobar Sistemas de recordatorio/restauración de contraseñas vulnerables
- ✓ Comprobar Sistemas de recordatorio/restauración de contraseñas vulnerables

- ✓ Pruebas de gestión del Caché de Navegación y de salida de sesión
- ✓ Pruebas de CAPTCHA
- ✓ Múltiples factores de autenticación
- ✓ Probar por situaciones adversas

e) Pruebas de Autorización

Autorización es el concepto de permitir el acceso a recursos únicamente a aquellos que tienen permiso para ello. Las pruebas de Autorización significan entender cómo funciona el proceso de autorización, y usar esa información para saltarse el mecanismo de autorización.

- ✓ Los usuarios que acceden a los recursos tienen credenciales válidas para hacerlo. Se deberá considerar validaciones mediante control de acceso vía Active Directory para el ingreso a los sistemas, este control solo será para saber que el usuario es válido, los perfiles y roles de control de la aplicación podrán seguir siendo controlados por el sistema.
- ✓ Los usuarios están asociados con un conjunto bien definido de roles y privilegios.
- ✓ Los metadatos de roles y permisos están protegidos contra la reproducción o manipulación.

f) Gestión de Sesiones

La gestión de sesiones cubre ampliamente todos los controles que se realizan sobre el usuario, desde la autenticación hasta la salida de la aplicación. HTTP es un protocolo sin manejo de estados, lo que significa que los servidores web responden a las peticiones de clientes sin enlazarlas entre sí.

Es importante que la seguridad de la aplicación sea considerada en el contexto de los requisitos y expectativas del proveedor.

Asegúrese de que una aplicación verificada satisface los siguientes requisitos de gestión de sesiones de alto nivel:

- ✓ Las sesiones son únicas para cada individuo y no pueden ser adivinadas o compartidas.
- ✓ Las sesiones se invalidan cuando ya no son necesarias y se anulan durante los períodos de inactividad.

g) Pruebas de Validación de Datos

La debilidad más común en la seguridad de aplicaciones web, es la falta de una validación adecuada de las entradas procedentes del cliente o del entorno de la aplicación. Esta debilidad conduce a casi todas las principales vulnerabilidades en aplicaciones, como inyecciones sql sobre campos de control de acceso, ataques locale/Unicode, sobre el sistema de archivos y desbordamientos de búfer.

Asegúrese de que una aplicación verificada satisface los siguientes requisitos de alto nivel:

- ✓ Todas las entradas están validadas para ser utilizables para el propósito previsto.
- ✓ Los datos de una entidad externa o cliente nunca deben ser confiables y deben tratarse, en consecuencia, SIEMPRE.

h) Pruebas de Denegación de Servicio

El tipo más común de ataque de Denegación de Servicio (Dos, por sus siglas en inglés) es del tipo empleado en una red para hacer inalcanzable a la comunicación a un servidor por parte de otros usuarios válidos. El concepto fundamental de un ataque DoS de red es un usuario malicioso inundando con suficiente tráfico una máquina objetivo para conseguir hacerla incapaz de sostener el volumen de peticiones que recibe. Cuando el usuario malicioso emplea un gran número de máquinas para inundar de tráfico una sola máquina objetivo, se conoce generalmente como ataque denegación de servicios distribuidos (DDoS, por sus siglas en inglés).

i) Pruebas de Servicios WEB

Los servicios web y SOA (Arquitectura Orientada a Servicios) son aplicaciones en expansión que están permitiendo que los negocios interoperen y crezcan a un ritmo sin precedentes. Los clientes de servicios web generalmente no son frontales web, sino otros servidores. Los servicios web están expuestos a la red como cualquier otro servicio, pero pueden ser utilizados en HTTP, FTP, SMTP o acompañados de cualquier otro protocolo de transporte.

Las vulnerabilidades en servicios web son similares a otras vulnerabilidades como la inyección SQL, revelación de información, etc., pero también tienen vulnerabilidades de XML.

j) Pruebas de AJAX

El uso de las técnicas AJAX puede conseguir enormes beneficios en la experiencia de uso por parte de los usuarios de las aplicaciones web. Sin embargo, desde el punto de vista de la seguridad, las aplicaciones AJAX tienen una superficie de ataque mayor que las aplicaciones web convencionales, a veces son desarrolladas centrándose más en qué se puede hacer que en qué se debería hacer. Además, las aplicaciones AJAX son más complicadas porque el procesamiento se realiza tanto en el lado del cliente como en el lado del servidor.

k) Pruebas de Criptografía

Debemos asegurarnos de que una aplicación verificada satisface los siguientes requisitos de alto nivel:

- ✓ Que todos los módulos criptográficos cuando fallan lo hacen de forma segura y que los errores son manejados correctamente.
- ✓ Que se utilice un generador de números aleatorios adecuado cuando se requiera aleatoriedad.
- ✓ Que el acceso a las llaves criptográficas se gestione de forma segura.

l) Manejo de errores

El objetivo principal del manejo y registro de errores es proporcionar una reacción útil por parte del usuario, administradores y equipos de respuesta a incidentes. El objetivo no es crear cantidades masivas de registros, sino registros de alta calidad.

Los registros de alta calidad a menudo contienen datos confidenciales y deben estar protegidos según las leyes o directivas locales de privacidad de datos. Esto debe incluir:

- ✓ No recopilar o registrar información confidencial si no se requiere específicamente.
- ✓ Asegurar que toda la información registrada sea manejada de forma segura y protegida según su clasificación de datos.
- ✓ Tener presente que, si los registros contienen datos privados o confidenciales, cuya definición varía de un país a otro, los registros se convierten en la información más sensible que posee la aplicación, por lo que resultan muy atractivos para los atacantes.

m) Protección de datos

Hay tres elementos claves que debemos gestionar para una sólida protección de datos: Confidencialidad, Integridad y Disponibilidad (CID). Este estándar supone que la protección de datos se aplica en un sistema fiable, como un servidor, que ha sido reforzado y tiene suficientes protecciones. Las aplicaciones tienen que asumir que todos los dispositivos del usuario están comprometidos de alguna u otra manera. Cuando una aplicación transmite o almacena información confidencial en dispositivos inseguros, como ordenadores, teléfonos y tabletas compartidos, la aplicación es responsable de garantizar que los datos almacenados en estos dispositivos estén cifrados y no puedan obtenerse, alterarse o divulgarse fácilmente de forma ilícita.

Asegúrese de que una aplicación verificada satisface los siguientes requisitos de protección de datos de alto nivel:

- ✓ Confidencialidad: Los datos deben protegerse de la observación o divulgación no autorizadas tanto en tránsito como almacenados.
- ✓ Integridad: Los datos deben estar protegidos al ser creados, alterados o borrados maliciosamente por atacantes no autorizados.
- ✓ Disponibilidad: Los datos deben estar disponibles para los usuarios autorizados según sea necesario.

n) Código malicioso

Debemos estar seguros de que una aplicación verificada satisface los siguientes requisitos de alto nivel:

- ✓ La posible actividad maliciosa puede administrarse de forma segura y correcta para no afectar al resto de la aplicación.

- ✓ No tienen malware que se activen por tiempo u otros ataques basados en factores temporales incorporados en ellas.
- ✓ No realice llamadas a destinos maliciosos o no autorizados.
- ✓ Las aplicaciones no tienen puertas traseras (backdoors) o fallas lógicas que pueden ser controladas por un atacante.

El código malicioso es extremadamente raro y difícil de detectar. La revisión manual de código línea por línea puede ayudar a buscar malware inserto en el código, pero incluso el revisor de código más experimentado luchará por encontrar código malicioso, aunque sepa que existe. Esta sección no es posible completarla sin acceso al código fuente, incluyendo tantas bibliotecas de terceros como sea posible.

6.4 ESTANDARES DE CODIFICACION SEI CERT A CONSIDERAR

6.4.1 Validar la entrada

Valide la entrada de todas las fuentes de datos que no sean de confianza. La validación de entrada adecuada puede eliminar la gran mayoría de las vulnerabilidades de software. Sospeche de la mayoría de las fuentes de datos externas, incluidos los argumentos de la línea de comandos, las interfaces de red, las variables de entorno y los archivos controlados por el usuario.

6.4.2 Prestar atención a las advertencias del compilador

Compile el código utilizando el nivel de advertencia más alto disponible para su compilador y elimine las advertencias modificando el código. Utilice herramientas de análisis estático y dinámico para detectar y eliminar fallas de seguridad adicionales.

6.4.3 Arquitectura y diseño de políticas de seguridad

Cree una arquitectura de software y diseñe su software para implementar y hacer cumplir las políticas de seguridad. Por ejemplo, si su sistema requiere diferentes privilegios en diferentes momentos, considere la posibilidad de dividir el sistema en distintos subsistemas de intercomunicación, cada uno con un conjunto de privilegios adecuados.

6.4.4 Mantenlo simple

Mantenga el diseño lo más simple y pequeño posible. Los diseños complejos aumentan la probabilidad de que se cometan errores en su implementación, configuración y uso. Además, el esfuerzo requerido para lograr un nivel apropiado de seguridad aumenta dramáticamente a medida que los mecanismos de seguridad se vuelven más complejos.

6.4.5 Por defecto denegar

Basar las decisiones de acceso en el permiso, en lugar de la exclusión. Esto significa que, de forma predeterminada, el acceso está denegado y el esquema de protección identifica las condiciones bajo las cuales se permite el acceso.

6.4.6 Cumplir con el principio de privilegio mínimo

Cada proceso debe ejecutarse con el mínimo conjunto de privilegios necesarios para completar el trabajo. Solo se debe acceder a cualquier permiso elevado durante el menor tiempo necesario para completar la tarea privilegiada. Este enfoque reduce las oportunidades que tiene un atacante para ejecutar código arbitrario con privilegios elevados.

6.4.7 Sanitizar los datos enviados a otros sistemas

Sanitice todos los datos pasados a los subsistemas complejos, como los shells de comando, las bases de datos relacionales y los componentes comerciales disponibles. Los atacantes pueden invocar la funcionalidad no utilizada en estos componentes mediante el uso de SQL, comandos u otros ataques de inyección. Este no es necesariamente un problema de validación de entrada porque el complejo subsistema que se invoca no comprende el contexto en el que se realiza la llamada. Debido a que el proceso de llamada comprende el contexto, es responsable de limpiar los datos antes de invocar el subsistema.

6.4.8 Practicar la defensa en profundidad

Administre el riesgo con múltiples estrategias de defensa, de modo que si una capa de defensa resulta inadecuada, otra capa de defensa puede evitar que una falla de seguridad se convierta en una vulnerabilidad explotable o limite las consecuencias de una explotación exitosa. Por ejemplo, la combinación de técnicas de programación segura con entornos de tiempo de ejecución seguros debería reducir la probabilidad de que las vulnerabilidades que permanecen en el código en el momento del despliegue se puedan explotar en el entorno operativo.

6.4.9 Usar técnicas efectivas de aseguramiento de la calidad

Las buenas técnicas de garantía de calidad pueden ser efectivas para identificar y eliminar vulnerabilidades. Las pruebas de Fuzz, las pruebas de penetración y las auditorías de código fuente deben incorporarse como parte de un programa de control de calidad efectivo. Las revisiones de seguridad independientes pueden conducir a sistemas más seguros. Los revisores externos aportan una perspectiva independiente; por ejemplo, en la identificación y corrección de suposiciones no válidas.

6.4.10 Adoptar una norma de codificación segura

Desarrolle o aplique un estándar de codificación seguro para su plataforma y lenguaje de desarrollo de destino.

6.4.11 Definir los requisitos de seguridad

Identifique y documente los requisitos de seguridad al inicio del ciclo de vida del desarrollo y asegúrese de que los artefactos de desarrollo subsiguientes se evalúen para cumplir con esos requisitos. Cuando no se definen los requisitos de seguridad, la seguridad del sistema resultante no se puede evaluar de manera efectiva.

6.4.12 Modelo de amenazas

Utilice el modelamiento de amenazas para anticipar las amenazas a las que estará sujeto el software. El modelamiento de amenazas implica identificar activos clave, descomponer la aplicación, identificar y categorizar las amenazas para cada activo o componente, calificar las amenazas según una clasificación de riesgo y luego desarrollar estrategias de mitigación de amenazas que se implementan en diseños, códigos y casos de prueba.

6.5 OWASP PROYECTO DE CICLO DE VIDA DE DESARROLLO SEGURO (S-SDLC)

El Proyecto de Ciclo de Vida de Desarrollo de Software Seguro (S-SDLC) de OWASP es una metodología de software de seguridad general para desarrolladores de aplicaciones y aplicaciones web. Su objetivo es definir un ciclo de vida de desarrollo de software seguro estándar y luego ayudar a los desarrolladores a saber qué deben considerarse o las mejores prácticas en cada fase de un Ciclo de vida de desarrollo (por ejemplo, Fase de diseño / Fase de codificación / Fase de mantenimiento / etc.).

La seguridad del software ahora se ha convertido en un concepto más amplio que no es la seguridad de la red. Se está desarrollando un sentido común de que crear un software suficientemente seguro no se trata solo de habilidades individuales, sino también o incluso más en los flujos de trabajo: ciclo de vida del desarrollo de software. Para lograr la seguridad, es necesario participar en cada fase de un ciclo de vida de desarrollo de software seguro.

El objetivo final del proyecto es ayudar a los usuarios a reducir los problemas de seguridad y elevar el nivel de seguridad general de cada etapa mediante el uso de la metodología.

Referencia:

[https://www.owasp.org/index.php/OWASP_Secure_Software_Development_Lifecycle_Proj](https://www.owasp.org/index.php/OWASP_Secure_Software_Development_Lifecycle_Project)
[ecthttps://info.veracode.com/secure-coding-best-practices-hand-book-guide-resource.html](https://info.veracode.com/secure-coding-best-practices-hand-book-guide-resource.html)

6.6 LAS 10 MEJORES PRÁCTICAS DE OWASP

6.6.1 Inyección

Las fallas de inyección, como SQL, NoSQL, OS o LDAP ocurren cuando se envían datos no confiables a un intérprete, como parte de un comando o consulta. Los datos dañinos del atacante pueden engañar al intérprete para que ejecute comandos involuntarios o acceda a los datos sin la debida autorización.

6.6.2 Pérdida de Autenticación

Las funciones de la aplicación relacionadas a autenticación y gestión de sesiones son implementadas incorrectamente, permitiendo a los atacantes comprometer usuarios y contraseñas, token de sesiones, o explotar otras fallas de implementación para asumir la identidad de otros usuarios (temporal o permanentemente).

6.6.3 Exposición de datos sensibles

Muchas aplicaciones web y APIs no protegen adecuadamente datos sensibles, tales como información financiera, de salud o Información Personalmente Identificable (PII). Los atacantes pueden robar o modificar estos datos protegidos inadecuadamente para llevar a cabo fraudes con tarjetas de crédito, robos de identidad u otros delitos. Los datos sensibles requieren métodos de protección adicionales, como el cifrado en almacenamiento y tránsito.

6.6.4 Entidades Externas XML (XXE)

Muchos procesadores XML antiguos o mal configurados evalúan referencias a entidades externas en documentos XML. Las entidades externas pueden utilizarse para revelar archivos internos mediante la URI o archivos internos en servidores no actualizados, escanear puertos de la LAN, ejecutar código de forma remota y realizar ataques de denegación de servicio (DoS).

6.6.5 Pérdida de Control de Acceso

Las restricciones sobre lo que los usuarios autenticados pueden hacer no se aplican correctamente. Los atacantes pueden explotar estos defectos para acceder, de forma no autorizada, a funcionalidades y/o datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar derechos de acceso y permisos, etc.

6.6.6 Configuración de Seguridad Incorrecta

La configuración de seguridad incorrecta es un problema muy común y se debe en parte a establecer la configuración de forma manual, ad hoc o por omisión (o directamente por la falta de configuración). Son ejemplos: S3 buckets abiertos, cabeceras HTTP mal configuradas, mensajes de error con contenido sensible, falta de parches y actualizaciones, frameworks, dependencias y componentes desactualizados, etc.

6.6.7 Secuencia de Comandos en Sitios Cruzados (XSS)

Los XSS ocurren cuando una aplicación toma datos no confiables y los envía al navegador web sin una validación y codificación apropiada; o actualiza una página web existente con datos suministrados por el usuario utilizando una API que ejecuta JavaScript en el navegador. Permiten ejecutar comandos en el navegador de la víctima y el atacante puede secuestrar una sesión, modificar (defacement) los sitios web, o redireccionar al usuario hacia un sitio malicioso.

6.6.8 Deserialización Insegura

Estos defectos ocurren cuando una aplicación recibe objetos serializados dañinos y estos objetos pueden ser manipulados o borrados por el atacante para realizar ataques de repetición, inyecciones o elevar sus privilegios de ejecución. En el peor de los casos, la deserialización insegura puede conducir a la ejecución remota de código en el servidor.

6.6.9 Componentes con vulnerabilidades conocidas

Los componentes como bibliotecas, frameworks y otros módulos se ejecutan con los mismos privilegios que la aplicación. Si se explota un componente vulnerable, el ataque puede provocar una pérdida de datos o tomar el control del servidor. Las aplicaciones y API que utilizan componentes con vulnerabilidades conocidas pueden debilitar las defensas de las aplicaciones y permitir diversos ataques e impactos.

6.6.10 Registro y Monitoreo Insuficientes

El registro y monitoreo insuficiente, junto a la falta de respuesta ante incidentes permiten a los atacantes mantener el ataque en el tiempo, pivotar a otros sistemas y manipular, extraer o destruir datos. Los estudios muestran que el tiempo de detección de una brecha de seguridad es mayor a 200 días, siendo típicamente detectado por terceros en lugar de por procesos internos.

Referencia:

<https://www.owasp.org/images/5/5e/OWASP-Top-10-2017-es.pdf>

6.7 ENTORNOS DE DESARROLLO SEPARADOS DE PRODUCCIÓN

Los entornos de desarrollo, pruebas y operacionales se deben separar para reducir los riesgos del acceso o cambios no autorizados al entorno operacional.

Se debe identificar e implementar el nivel de separación entre los entornos operativos, de prueba y desarrollo necesario para evitar los problemas operacionales.

Se debe considerar los siguientes elementos:

- se deberían definir y documentar las reglas de transferencia de software desde un estado de desarrollo al operacional;
- el software de desarrollo y operativo se debería ejecutar en distintos sistemas o procesadores de computador y en distintos dominios y directorios;
- se deberían probar los cambios a los sistemas operativos y aplicaciones en un entorno de pruebas o etapas antes de aplicarlos a los sistemas operacionales;
- a no ser que sea bajo circunstancias excepcionales, no se deberían realizar pruebas en los sistemas basados en ambientes productivos;
- los compiladores, editores y otras herramientas de desarrollo o utilidades del sistema no deberían estar accesibles desde los sistemas operacionales cuando no sea necesario;
- los usuarios deberían utilizar distintos perfiles de usuario para los sistemas operacionales y de prueba y mostrar menús para mostrar mensajes de identificación adecuados para reducir el riesgo de errores de conexión a bases incorrectas.
- los datos sensibles no se deberían copiar en el entorno del sistema de pruebas a menos que se entreguen controles equivalentes para el sistema de pruebas.

Las actividades de desarrollo y pruebas pueden provocar problemas graves, es decir, la modificación no deseada de archivos o del entorno del sistema o una falla del sistema.

Existe la necesidad de mantener un entorno conocido y estable en el que se pueden realizar pruebas significativas para evitar el acceso inadecuado del desarrollador al entorno productivo.

Cuando el personal de desarrollo y pruebas tenga acceso al sistema operativo y a su información, podrían introducir un código no autorizado o no probado, o alterar los datos productivos. En algunos sistemas esta capacidad se podría utilizar incorrectamente para cometer fraudes o introducir un código sin probar o malicioso, lo que puede generar problemas operacionales graves.

El personal de desarrollo y pruebas también representa una amenaza a la confidencialidad de la información operacional. Las actividades de desarrollo y pruebas pueden provocar cambios no intencionados al software o la información si comparten el mismo entorno computacional. Por lo tanto, se aconseja la separación de los entornos desarrollo, de pruebas y operativos para reducir el riesgo de cambios accidentales o del acceso no autorizado al software operacional y los datos del negocio.

6.8 DOCUMENTACION REQUERIDA SOBRE DESARROLLOS

6.8.1 Modelo de Datos

Cada desarrollo deberá contar con la información de tablas o estructuras de datos utilizadas por el aplicativo, estas estructuras de datos conocidas como el modelo de datos, deberá quedar correctamente documentado.

La documentación requerida deberá considerar al menos la identificación de llaves primarias y foráneas y una descripción detallada (documentación en la tabla) de campos relevantes que deban ser considerados como especiales en el modelo de datos.

Es importante considerar ser lo más nemotécnico posible en la creación de los nombres de tablas y de campos en las correspondientes estructuras, cosa de evitar la documentación adicional de cada uno de ellos, la idea es que los nombres utilizados sean auto explicativos.

6.8.2 Modelo Entidad Relación

Cada desarrollo deberá contar con un modelo o diagrama de entidad relación que especifique como se relacionan las entidades del modelo de datos, este modelo deberá al menos mostrar las relaciones existentes a nivel de llave primaria y foránea. Esta documentación será de vital importancia para entender el alcance del desarrollo realizado. En este modelo deberá quedar especificada la cardinalidad de las relaciones entre las entidades.

6.8.3 Documentación de Sistemas (Diagrama de diseño Lógico)

El propósito de la documentación es enseñar a quienes no están familiarizados con un sistema cómo este se estructura, funciona y los motivos que llevaron a decidirse por ese diseño. Los principales usuarios de la documentación de diseño son los futuros responsables del mantenimiento del sistema. Por este motivo es de vital importancia que el tiempo definido para el desarrollo del proyecto considere tiempo de documentación. Por consiguiente, cada desarrollo deberá quedar claramente documentado mediante un

Diagrama de Diseño Lógico de las principales funciones del sistema y sus interrelaciones de cada una de las funciones con el modelo de datos.

6.8.4 Documentación de Requisitos Básicos

El propósito de esta documentación será establecer los estándares mínimos en los que podrá operar el desarrollo construido, especificando en este documento los requisitos mínimos para su correcta operación:

- Versión sistema operativo estación de trabajo
- Versión sistema operativo del servidor donde se alojará el aplicativo de ser requerido
- Versión sistema operativo del servidor donde se alojará la Base de Datos
- Versión de la Base de Datos
- Versión Navegador Internet de ser requerido
- Espacios de almacenamiento requeridos mínimos y estimación promedio de crecimiento anual.

6.8.5 Manuales de Usuarios

Cada desarrollo deberá contar con manuales de uso especificando el funcionamiento en detalle de cada función del desarrollo.

La estructura de este manual al menos deberá considerar lo siguiente:

- Requerimientos Básicos
- Roles
- Descripción de pantallas y sus objetivos
- Control de Errores
- Contingencia y Soporte Técnico

6.9 IMPLEMENTACION DE DESARROLLOS ADQUIRIDOS A TERCEROS

Es importante mencionar que adquisiciones o recepción de software no desarrollados en forma interna deben cumplir con los requisitos de compatibilidad necesarios de acuerdo a la arquitectura de la plataforma de desarrollo definida en el punto 6.1 y cumplir con la documentación de sistema especificada en el punto 6.8 del presente documento de no cumplir con esta norma se deberá coordinar previamente con el proveedor la implementación de dicha compatibilidad.

6.10 REQUISITO PARA PASO A PRODUCCION

Cualquier desarrollo interno o sistema adquirido que se necesite instalar en el ambiente productivo de la organización, deberá primero cumplir los siguientes requisitos:

- Deberá ser testeado y certificado por el área de QA. Esta área será la encargada de implementar el mejor método o estrategia de prueba que considere necesario. Es importante mencionar que independiente del método utilizado cada una de las pruebas realizadas deberán ser documentadas y entregadas a desarrollo y operaciones con sus correspondientes resultados. Por consiguiente, el área de QA,

será la encargada de certificar el correcto funcionamiento de cada software que se requiera implementar en el ambiente de producción.

- Deberá ser testeado y aprobado por el área de Seguridad, quién realizará pruebas para identificar posibles vulnerabilidades.
- Deberá ser entregado con toda la documentación completa del sistema especificada en el punto 8 del presente documento.
- Deberá ser entregado toda la documentación legal en el caso de tratarse de una adquisición (Factura de compra, licenciamiento, etc.).
- Deberá ser entregado la versión actualizada de su código fuente y su correspondiente documentación para almacenamiento digital.

7 REGISTROS

- Documentos de desarrollo
 - ✓ Modelo de Datos
 - ✓ Modelo Entidad Relación
 - ✓ Documentación de Sistemas (Diagrama de diseño Lógico)
 - ✓ Documentación de Requisitos Básicos
 - ✓ Manuales de Usuarios
- Registros de pruebas de aceptación de sistemas según lo especificado en el punto 6.3.3 de este procedimiento

8 DIFUSION

La comunicación del presente procedimiento se efectuará de manera que el contenido de la documentación sea accesible y comprensible para todos los usuarios, a lo menos se deberá hacer difusión mediante los siguientes canales:

- Publicación en la intranet de Minsal <http://isalud.minsal.cl/>
- Correo informativo.

9 REVISION Y MEDICION

El presente procedimiento deberá ser revisado a lo menos cada dos años o cuando ocurran cambios significativos para asegurar su continua idoneidad, eficiencia y efectividad.

10 CONTROL DE VERSIONES

Versión	Fecha de Aprobación	Motivo del cambio	Secciones modificadas
01	Octubre 2019	Creación del documento	Todas

11 ANEXOS

Anexo 1: directrices ISO 27001 a cumplir

- Nch-ISO27002 Control 7.1.1 Selección
- Nch-ISO27002 Control 12.1.1 Procedimientos operativos documentados
- Nch-ISO27002 Control 12.1.2 Administración de cambios
- Nch-ISO27002 Control 12.1.4 Separación de entornos de desarrollo, pruebas y operacionales
- Nch-ISO27002 Control 12.6.1 Administración de vulnerabilidades técnicas
- Nch-ISO27002 Control 14.1.1 Análisis y especificación de los requisitos de seguridad de la información
- Nch-ISO27002 Control 14.1.2 Protección de servicios de aplicación en redes públicas
- Nch-ISO27002 Control 14.1.3 Protección de transacciones de servicios de aplicación
- Nch-ISO27002 Control 14.2.1 Política de desarrollo seguro
- Nch-ISO27002 Control 14.2.2 Procedimientos de control de cambios del sistema
- Nch-ISO27002 Control 14.2.3 Revisión técnica de las aplicaciones después de los cambios en la plataforma operativa
- Nch-ISO27002 Control 14.2.4 Restricciones a los cambios de paquetes de software
- Nch-ISO27002 Control 14.2.5 Principios de ingeniería segura del sistema
- Nch-ISO27002 Control 14.2.6 Entorno de desarrollo seguro
- Nch-ISO27002 Control 14.2.7 Desarrollo externalizado
- Nch-ISO27002 Control 14.2.8 Pruebas de seguridad del sistema
- Nch-ISO27002 Control 14.3.1 Protección de los datos de pruebas
- Nch-ISO27002 Control 18.1.2 Derechos de propiedad intelectual
- Nch-ISO27002 Control 18.1.5 Regulación de controles criptográficos
- Complementariamente ISO 27036 : Information security for supplier relationships (four parts)
- Complementariamente ISO 29101 : Privacy architecture framework

Anexo 2: listado de herramientas que pueden sr utilizadas para test de aplicaciones

1. Selenium (Web Application Testing)
2. Appium (Mobile Testing)
3. JMeter (Load Testing)
4. Jenkins (Continuous Testing)
5. TestLink (Test Management)
6. Mantis (Bug-Tracking & Project Management)
7. Postman (API Testing)
8. Firebug / Firepath (Online Debugging)
9. GitHub (Project & Source Code Hosting)
10. Bugzilla (Defect Tracking & Collaboration)
11. RazorSQL (Database Query Tool)
12. PhantomJS (Headless Browser)
13. UIAutomator (Android Testing Framework)

- 14. Notepad++ (Source code Editor)
- 15. FileZilla (FTP Solution)
- 16. AutoIT (Language Automation)

Selenium

Selenium es uno de los frameworks más utilizados para probar aplicaciones web, principalmente para la interfaz web y las pruebas funcionales. Viene con una serie de herramientas como Selenium IDE, Selenium RC, Selenium WebDriver y Selenium Grid que ofrece diferentes soluciones para atender diferentes requisitos de automatización de pruebas.

Referencias <http://www.seleniumhq.org/projects/webdriver/>

Appium

Appium es un framework de automatización de pruebas para probar aplicaciones web nativas, híbridas y móviles para plataformas iOS, Android y Windows en dispositivos reales y simuladores. Dado que soporta aplicaciones multiplataforma, permite probar aplicaciones en diferentes plataformas utilizando la misma API. Appium permite a los usuarios elegir el idioma que tiene las bibliotecas de clientes de Selenium como Java, Objective-C, JavaScript con Node.js, PHP, Ruby, Python, C # etc. para crear pruebas.

Referencias <http://appium.io>

JMeter

JMeter es una herramienta basada en Java diseñada para cargar el comportamiento de la aplicación y medir el rendimiento del sitio web. Puede probar recursos estáticos y dinámicos que incluyen servicios web SOAP / REST, sitios web HTTP y HTTPS, bases de datos, FTP y servidores de correo, así como PHP, ASP.NET y Java. Funciona simulando la carga en el servidor para analizar el rendimiento general de la aplicación / sitio web bajo prueba.

Referencias <http://jmeter.apache.org>

Jenkins

Jenkins es una herramienta para iniciar pruebas continuas y construir la integración a través de la automatización. Proporciona una forma poderosa de administrar los cambios de código, las pruebas y el ciclo de vida del despliegue, junto con la administración de releases, acelerando el ciclo de vida general del desarrollo del software. Hoy en día, Jenkins ofrece soporte para más de 1.200 plugins que le permiten integrarse con cualquier tecnología.

Referencias <https://jenkins.io>

Testlink

TestLink es una herramienta de gestión de pruebas basada en la web ampliamente utilizada. Proporciona soporte para administrar y mantener casos de prueba, conjuntos de pruebas, documentos de prueba y proyectos en un solo lugar. Puede alojarse en un servidor e integrado con herramientas de seguimiento de errores como Mantis, JIRA, Bugzilla, FogBugz, etc. para facilitar el proceso de ejecución de pruebas. TestLink se puede utilizar tanto para pruebas manuales como automatizadas.

Referencias <http://testlink.org>

Mantis

Mantis es una herramienta líder de seguimiento de errores utilizada por los probadores para el seguimiento de errores encontrados en el software durante el proceso de prueba. También proporciona funciones de gestión de proyectos y administración de problemas que ayudan a lograr una colaboración más rápida y efectiva entre equipos y clientes.

Referencias <https://www.mantisbt.org>

Postman

Postman es una gran herramienta para probar APIs. Los probadores y desarrolladores pueden utilizar esta herramienta gratuita como una extensión de Chrome o un producto de colaboración en la nube para desarrollar, probar y documentar las API más rápidamente. Permite a los usuarios comprobar el historial de las solicitudes HTTP enviadas, personalizar secuencias de comandos, autocompletar URL, previsualizar imágenes, realizar pruebas de producción, organizar o configuraciones locales con una amplia gama de características y funciones.

Referencias www.getpostman.com

Firebug

Firebug es una extensión de navegador web que ayuda a los probadores en la depuración, edición y supervisión en línea de CSS, HTML y JavaScript de la aplicación web. Firebug junto con Firepath se utiliza para identificar XPath de cualquier elemento. Tanto Firebug como Firepath pueden instalarse como una extensión para Mozilla Firefox, mientras que Firebug Lite se puede agregar como una extensión de Chrome que proporciona una rica presentación de elementos HTML y DOM para la edición en directo.

Referencias <http://getfirebug.com>

GitHub

GitHub es un servicio de repositorio basado en la web para alojar y administrar proyectos de software, versiones y código fuente. Proporciona características como edición en línea, ticketing, seguimiento de errores, administración de tareas, así como funciones de redes sociales como feed, wikis, que ayudan a millones de desarrolladores y probadores a trabajar de manera colaborativa. Promueve el desarrollo rápido y flexible de proyectos con más de 14 millones de usuarios y más de 35 millones de repositorios.

Referencias <https://github.com>

Bugzilla

Bugzilla es otra herramienta de rastreo y prueba de defectos que es ampliamente utilizada por los probadores para realizar un seguimiento de los errores pendientes. Viene con una variedad de características tales como un sistema integrado del email, gerencia avanzada de la pregunta, sistema de los permisos, el sistema incorporado del informe así como los perfiles editable del usuario para asegurar proceso de prueba liso y eficaz.

Referencias <https://www.bugzilla.org>

Razor SQL

Razor SQL es una herramienta de SQL Query y Database Editor para Windows, Mac OS y Linux. Permite a los probadores importar, exportar y convertir bases de datos en varios formatos como MySQL, Oracle, DB2, PostgreSQL, SQLite, MS SQL Server y MS Access. Con Razor SQL, los usuarios también pueden explorar objetos de base de datos y realizar comparaciones de bases de datos.

Referencias <https://razorsql.com>

PhantomJS

PhantomJS es un navegador que se utiliza para automatizar las interacciones de la página con fines de prueba. Ayuda a los usuarios a habilitar la navegación y el comportamiento del usuario en una página sin cargar la interfaz gráfica. PhantomJS imita y manipula una página web para llevar a cabo la automatización de pruebas que en última instancia, ahorra una tremenda cantidad de tiempo para los probadores.

Referencias <http://phantomjs.org>

UIAutomator

UIAutomator es un marco para pruebas de interfaz de usuario funcional para aplicaciones de Android. Permite a los probadores probar las aplicaciones de Android creando múltiples casos de prueba que pueden ejecutarse en varios dispositivos con diferentes resoluciones. UIAutomator también puede utilizarse para probar aplicaciones preinstaladas, como Ajustes del teléfono, así como aplicaciones de terceros.

Referencias <https://google.github.io/android-testing-support-library/docs/uiautomator/>

Notepad++

Notepad ++ es un editor de texto que permite a los usuarios editar el código fuente de 27 lenguajes de programación en entorno Windows. También soporta resaltado de sintaxis y plegado, ediciones sincronizadas, zoom in y zoom out, vistas múltiples, marcadores, grabación de macros y reproducción junto con GUI personalizable.

Referencias <https://notepad-plus-plus.org>

FileZilla

FileZilla es una aplicación FTP multiplataforma para clientes y servidores. Permite a los usuarios cargar y descargar archivos desde y hacia su sitio FTP, así como realizar múltiples transferencias de archivos y navegación simultáneamente. FileZilla ayuda a transferir en FTP, SFTP, FTP cifrado como FTPS y SFTP. También incluye un administrador de sitio que puede almacenar todos los detalles de la conexión en una interfaz tipo Explorer.

Referencias <https://filezilla-project.org>

AutoIT

AutoIT es una herramienta para automatizar la GUI de Windows y las secuencias de comandos generales usando una combinación de pulsaciones de teclas, movimiento del ratón y manipulación de ventana / control. Se utiliza para automatizar tareas que son difíciles de realizar con ciertos idiomas. Es muy utilizado por los probadores para crear scripts de automatización para el entorno de Windows.

Referencias: <https://es.m.wikipedia.org/wiki/AutoIt>